

Primary sources & further reading

Track 16.6 — AI on AWS with Amazon Bedrock · 16-ai-tech-stack/tracks/aws-bedrock/papers/PAPERS.md · 16-ai-tech-stack/tracks/aws-bedrock/resources/RESOURCES.md



Primary Sources — Amazon Bedrock

There is no arXiv paper for a cloud API. The primary sources here are the AWS documentation pages themselves, and reading them properly is the skill.

Every page below was fetched and verified in September 2026. AWS ships weekly; if a page contradicts this track, the page is right. When you find a discrepancy, fix the track (Exercise 9).

Read these three first

#	Page	Why it matters
1	Inference using the Converse API	The single most important page. Full request and response shapes, every ContentBlock type, <code>inferenceConfig</code> , the ConverseStream event ordering diagram. Read it twice; write the shapes out from memory the second time.
2	Use a tool to complete a model response and Client-side tool use	The tool protocol, with a complete working Python loop. Note the three modes now documented — client-side, server-side (Lambda / AgentCore Gateway), and Anthropic-defined tool types.
3	Scaling and throughput best practices	The page that separates people who have run Bedrock in production from people who have not. Endpoint quota differences, 429 vs 503 vs 529, the ramp-up procedure, the exact botocore retry config.

The API surface

Page	Takeaway
What is Amazon Bedrock	Five quickstarts side by side — Messages, Responses, Chat Completions, Converse, InvokeModel. Useful precisely because it shows how many front doors now exist.
Models at a glance	The live model catalogue. Model IDs, per-model feature support, Regions. Never hardcode an ID you have not checked here.
Request access to models	Access is on by default with Marketplace permissions — but Anthropic models need a one-time use-case form. The 15-minute auto-subscription window explains a class of confusing <code>AccessDeniedException</code> .
Inference profiles	System-defined vs application profiles; application profiles are how you tag Bedrock costs per team.
Cross-Region inference · Geographic	Geographic vs global, the ~10% global discount, and the IAM policy that must list every destination Region.
Prompt caching	Implicit vs explicit, <code>cachePoint</code> syntax, per-model minimums, the <code>tools -> system -> messages</code> ordering rule, and the <code>inputTokens</code> accounting trap.
Provisioned Throughput	Model Units, commitment terms, and the line most people miss: inference profiles do not support PT.
Batch inference	Discounted async via S3 JSONL — with no tool calling and no structured output.
Troubleshooting API error codes	Which errors to retry, which never to. Also the 350-second idle-connection trap, which costs teams a day each time.
Quotas	Token-based quotas, tracked separately per endpoint.

RAG and agents

Page	Takeaway
Knowledge Bases	Managed vs customer-managed. Read the Managed KB section carefully — agentic retrieval, ACL filtering, AgentCore Gateway integration are all recent.
How content chunking works	Default, fixed-size, hierarchical, semantic, none. The hierarchical section explains why you get fewer results than you asked for.
Vector store prerequisites	Every supported store with its setup steps. The <code>faiss-not-nmslib</code> requirement for OpenSearch metadata filtering is buried here and will bite you.
Configure queries and response generation	<code>numberOfResults</code> , <code>overrideSearchType</code> , the full metadata-filter operator table, implicit filtering, reranking, query decomposition.
Bedrock Agents	Read the note at the top. Agents is now "Agents Classic" and closed to new customers. This one paragraph invalidates most Bedrock agent tutorials written before 2026.
What is AgentCore	The replacement: Runtime, Gateway, Memory, Identity, Code Interpreter, Browser, Observability, Policy, Evaluations. Framework-agnostic by design.

Safety, security and compliance

Page	Takeaway
Guardrails · Create your guardrail	All six policy types and their configuration fields.
Guardrails with the Converse API	<code>guardrailConfig</code> , <code>guardContent</code> , <code>stopReason: "guardrail_intervened"</code> , the trace shape — and the table showing that tool results, tool definitions and generated tool arguments are not evaluated. That table is the most security-relevant thing in the Bedrock docs.
Contextual grounding check	Grounding vs relevance, the 0–0.99 threshold range, the <code>grounding_source</code> / <code>query</code> qualifiers, and the four worked grounded/relevant permutations.
Data protection	The Model Deployment Account architecture — the concrete reason model providers cannot see your prompts.
Interface VPC endpoints	Every endpoint service name, the FIPS variants and their Regions, and endpoint policies.
Identity-based policy examples	Allow, deny, provisioned-model and console-minimum policies.
Model invocation logging	Off by default. Full log schema, the 100 KB inline limit, and the <code>identity.arn</code> attribution query.
Evaluate Bedrock resources	Programmatic, human, judge-model and RAG evaluation.
Amazon Bedrock pricing	The only rates you should ever quote. Bookmark it; do not memorise it.

Background reading worth the time

Source	Why
Timeouts, retries and backoff with jitter — Amazon Builders' Library, Marc Brooker	Why <i>full</i> jitter beats backoff-plus-noise. Short, canonical, and it explains the botocore retry formula you are configuring.
Retry with backoff pattern	The AWS prescriptive-guidance version, with the decision tree for retryable vs not.
Boto3 retries guide	Legacy vs standard vs adaptive. The default is <code>legacy</code> ; you want <code>standard</code> .
AWS Well-Architected — Generative AI Lens	AWS's own opinion on GenAI architecture, mapped to the six pillars. Read it critically — it is a vendor document — but the operational-excellence section is genuinely good.
Model Context Protocol specification	AgentCore Gateway speaks MCP. Module 09 covers it; this is the source.

How to read an AWS service doc (it is not like reading a paper)

1. **Start at the API reference, not the user guide.** The user guide tells you what a feature is for; the API reference tells you what the JSON actually looks like. You need the second.
2. **Read the Notes and Important callouts first.** That is where the limitations live. "Batch inference does not support tool calling" is a Note. "Agents is closed to new customers" is a Note.
3. **Check the date on everything else.** A blog post from 2024 describing Bedrock Agents is not wrong about 2024. It is wrong about today.
4. **Verify every model ID and every quota against your own account.** `aws bedrock list-foundation-models`, `aws bedrock list-inference-profiles`, and the Service Quotas console are ground truth. Documentation is a snapshot.
5. **Keep a one-paragraph note per page you rely on**, with the date you read it. In six months the note tells you what to re-check.

Resources — AI on AWS with Amazon Bedrock

Real URLs only. Anything undated should be assumed stale until you check it against docs.aws.amazon.com.

Official documentation (start here)

Doc	What it is
Amazon Bedrock User Guide	The main guide. Concepts, features, walkthroughs.
Amazon Bedrock API Reference	The JSON shapes. <code>Converse</code> , <code>ConverseStream</code> , <code>ContentBlock</code> , <code>TokenUsage</code> — where you settle arguments.
Amazon Bedrock AgentCore Developer Guide	The current agent platform. Runtime, Gateway, Memory, Identity, Policy.
Amazon Bedrock pricing	The only place to get rates.
Service Quotas for Bedrock	Endpoints and default quotas per Region.
Service Authorization Reference — Bedrock	Every IAM action, resource type and condition key. Use it when writing least-privilege policies.
AWS Well-Architected Generative AI Lens	AWS's architectural opinion, mapped to the six pillars.
Boto3 bedrock-runtime reference	Python method signatures for <code>converse</code> , <code>converse_stream</code> , <code>apply_guardrail</code> .
Boto3 bedrock-agent-runtime reference	<code>retrieve</code> , <code>retrieve_and_generate</code> , and the agent runtime calls.

Repositories worth cloning

Repo	What's inside
https://github.com/aws-samples/amazon-bedrock-workshop	The official hands-on workshop. Notebooks for Converse, tool use, Knowledge Bases, Guardrails. The fastest path from reading to running.
https://github.com/awsdocs/aws-doc-sdk-examples	Canonical SDK examples across languages. <code>python/example_code/bedrock-runtime/</code> is the one you want; these are the snippets the docs embed.
https://github.com/aws-samples/amazon-bedrock-samples	Broader sample collection — RAG patterns, agents, evaluation, fine-tuning. Quality varies by folder; check commit dates.
https://github.com/aws-samples/generative-ai-cdk-constructs	L3 CDK constructs for Bedrock Knowledge Bases, agents and guardrails. The fastest way to get RAG infrastructure into IaC.
https://github.com/strands-agents/sdk-python	Strands Agents — AWS's open-source agent framework, a first-class citizen on AgentCore Runtime.
https://github.com/hashicorp/terraform-provider-aws	Check <code>website/docs/r/bedrock*</code> for which Bedrock resources are actually covered before you plan a Terraform module.
https://github.com/boto/boto3	When a parameter is not documented, the service model JSON in botocore is ground truth.
https://github.com/modelcontextprotocol/servers	Reference MCP servers. Relevant because AgentCore Gateway speaks MCP.

Courses and structured learning

Course	Who it's for	Link
AWS Skill Builder — Generative AI learning plans	Free digital courses, including Bedrock-specific paths	https://skillbuilder.aws/
Amazon Bedrock Workshop (self-paced, official)	The hands-on companion to this track	https://catalog.workshops.aws/
AWS Certified AI Practitioner (AIF-C01)	If you need the credential; heavy on Bedrock vocabulary	https://aws.amazon.com/certification/certified-ai-practitioner/
AWS Certified Machine Learning Engineer – Associate (MLA-C01)	The SageMaker-side certification; useful for the Bedrock-vs-SageMaker boundary	https://aws.amazon.com/certification/certified-machine-learning-engineer-associate/
DeepLearning.AI — Serverless LLM apps with Amazon Bedrock	Short, free, practical	https://www.deeplearning.ai/short-courses/

Blogs and ongoing sources

Source	Why follow it
AWS Machine Learning Blog	Where new Bedrock capabilities get explained with code, usually a day after launch.
What's New with AWS — Bedrock filter	The changelog. Skim weekly; it is how you learn a model was deprecated.
AWS Architecture Blog	Reference architectures, occasionally including honest post-mortems.
Amazon Builders' Library	Not Bedrock-specific and better for it. The retry, timeout and load-shedding articles are the intellectual basis for §3 of the production-craft note.
AWS re:Post — Amazon Bedrock	Where the errors nobody documented get answered. Search it before opening a support case.
AWS Service Terms and third-party model licences	The actual legal text your compliance team will ask for.

Communities

Where	Notes
AWS re:Post	Official Q&A; AWS staff answer. Higher signal than Stack Overflow for service behaviour.
r/aws	Good for "is this broken for everyone?" Poor for correctness.
AWS Community (community.aws)	Builder-written content; the Converse API developer guides for Java and JavaScript linked from the official docs live here.
MCP community	For the AgentCore Gateway side of agent work.

Tooling

Tool	Use
AWS CLI v2	<code>aws bedrock list-foundation-models</code> , <code>list-inference-profiles</code> , <code>get-foundation-model-availability</code> . Ground truth for what your account can actually call.
IAM Access Analyzer policy validation	Run every policy in this track through it before deploying.
AWS Cost Explorer + Budgets	Set the budget alarm before the first live call, not after the first surprise.
CloudWatch Logs Insights	Querying model invocation logs; see the <code>identity.arn</code> attribution query in the code folder.
LiteLLM	If you must abstract over Bedrock <i>and</i> other providers. Understand Converse first — an abstraction over an abstraction you do not understand is how you end up unable to debug anything.
LangChain ChatBedrockConverse	The LangChain binding that uses Converse rather than per-provider bodies. See the LangChain track [16-ai-tech-stack/tracks/langchain] in this module.

Sibling tracks in Module 16

Track	Relationship
LangChain [16-ai-tech-stack/tracks/langchain]	ChatBedrockConverse is the adapter; same tool-call semantics, different ergonomics
LangGraph [16-ai-tech-stack/tracks/langgraph]	Runs well on AgentCore Runtime; the graph is your loop
LangSmith [16-ai-tech-stack/tracks/langsmith]	Tracing alternative to CloudWatch + X-Ray for LLM-shaped spans
Python [16-ai-tech-stack/tracks/python] / TypeScript [16-ai-tech-stack/tracks/typescript]	The SDK ergonomics underneath all of this